



MCA SEM-I

SUBJECT: SOFTWARE ENGINEERING

UNIT- I

Introduction to software engineering

Software engineering is a framework for building software and is an engineering approach to software development.

Software programs can be developed without S/E principles and methodologies but they are indispensable if we want to achieve good quality software in a cost-effective manner.

Software is defined as:

Instructions + Data Structures + Documents

Engineering

Engineering is the branch of science and technology concerned with the design, building, and use of engines, machines, and structures. It is the application of science, tools and methods to find cost effective solution to simple and complex problems.

[Discover more](#)
[Software](#)
[Science](#)
[Networking](#)

Software engineering

Software engineering is defined as a systematic, disciplined and quantifiable approach for the development, operation and maintenance of software. It involves applying engineering principles to software creation to ensure that it is reliable, efficient, scalable, and meets the needs of users. This includes everything from understanding user requirements to writing code, testing it, and maintaining it over time.

Characteristics of software

- **Software is developed or engineered, but it is not manufactured in the classical sense.**
Development of software and other hardware involves manpower, but the quantity and the way of approach remain significantly different. In both the manufacturing processes, one will be left out with certain end products, but, the efforts applied in them remain different. Finally, both the activities are initiated with a determination of building high quality of products, but their quality maintaining activities differ. Hence, with these specifications it can be conclude that, “software is developed or engineered, it is not manufactured in the classical sense”.
- **Software does not wear out.**
One of the fundamental characteristics of software is that it does not wear out in the same way that physical objects do. Software doesn't degrade or lose functionality over time due to wear and tear. The hardware development processes the failure rate is very high because of manufacturing defects the failure rate remains constant for some period of time and again it starts increasing because of environment maladies (Extreme temperature, dusts, and Vibrations).

- **Software is custom built rather than assembling existing components.**
When software is custom-built rather than assembled from existing components, it means that the software is designed and developed specifically for a particular user, organization, or purpose, instead of being based on pre-made parts, libraries, or frameworks. Customizable software refers to software that can be modified or adjusted to meet the specific needs, preferences, or requirements of a user or organization. It allows users to change certain features, settings, or functionalities without needing to alter the underlying code. This flexibility makes customizable software suitable for a wide range of applications, as it can adapt to different contexts or user preferences.

The Evolving Role of Software

Software takes dual role. It is both a product and a vehicle for delivering a product.

As a product:

It delivers the computing potential embodied by computer hardware or by a network of computers.

As a vehicle:

It is information transformer—producing, managing, acquiring, modifying, displaying, or transmitting information that can be as simple as single bit or as complex as a multimedia presentation. Software delivers the most important product of our time—information.

- It transforms personal data
- It manages business information to enhance competitiveness
- It provides a gateway to worldwide information networks
- It provides the means for acquiring information
- The role of computer software has undergone significant change over a span of little more than 50 years
- Dramatic improvements in hardware performance
- Vast increases in memory and storage capacity
- A wide variety of exotic input and output options

Changing nature of software

The 7 broad categories of computer software present continuing challenges for software engineers:

- System software
- Application software
- Engineering/scientific
- Software embedded
- Software product-line
- Software web-applications
- Artificial intelligence software

System software

System software is a collection of programs written to service other programs. The systems software is characterized by heavy interaction with computer hardware, heavy usage by multiple users, concurrent operation that requires scheduling, resource sharing, and sophisticated process management, complex data structures, and multiple external interfaces.

E.g. compilers, editors and file management utilities.

Application software

Application software consists of standalone programs that solve a specific business need. It facilitates business operations or management/technical decision making. It is used to control business functions in real-time.

E.g. Point-of-sale transaction processing, real-time manufacturing process control.

Engineering/scientific software

Engineering and scientific applications range from astronomy to volcanology, from automotive stress analysis to space shuttle orbital dynamics, from molecular biology to automated manufacturing. **E.g.** computer aided design, system simulation and other interactive applications.

[Discover more](#)

[Protecting Online Personal Data Privacy](#)

[Sourcing Bulk Industrial Hardware data](#)

Embedded software

Embedded software resides within a product or system and is used to implement and control features and functions for the end-user and for the system itself. It can perform limited and esoteric functions or provide significant function and control capability. **E.g.** Digital functions in automobile, dashboard displays, braking systems etc.

Product-line software

Designed to provide a specific capability for use by many different customers, product-line software can focus on a limited and esoteric marketplace or address mass consumer markets. **E.g.** Word processing, spreadsheets, computer graphics, multimedia, entertainment, database management, personal and business financial applications.

[Discover more](#)

[Studying Advanced Computer Science Concepts](#)

[Purchasing Business Productivity Software](#)

[Hiring Civil Engineering Consultants](#)

Web-applications

WebApps are evolving into sophisticated computing environments that not only provide standalone features, computing functions, and content to the end user, but also are integrated with corporate databases and business applications.

Artificial intelligence software

AI software makes use of nonnumeric algorithms to solve complex problems that are not amenable to computation or straightforward analysis. Application within this area includes robotics, expert systems, pattern recognition, artificial neural networks, theorem proving, and game playing.

The following are the new challenges on the horizon

- Ubiquitous computing
- Net sourcing
- Open source
- The “new economy”

Ubiquitous computing

The challenge for software engineers will be to develop systems and application software that will allow small devices, personal computers and enterprise system to communicate across vast networks.

Net sourcing

The challenge for software engineers is to architect simple and sophisticated applications that provide benefit to targeted end-user market worldwide.

[Discover more](#)

[Balancing A Stocks And Bonds Portfolio](#)

Open source

The challenge for software engineers is to build source that is self-descriptive but more importantly to develop techniques that will enable both customers and developers to know what changes have been made and how those changes manifest themselves within the software.

The “new economy”

The challenge for software engineers is to build applications that will facilitate mass communication and mass product distribution.

Legacy software

Legacy software can be defined as old software developed in the past. This software is still being used in the present era as it performs essential business activities. It may include procedures which are no longer relevant in the newer computing environment. As business requirements are dynamic, the legacy software system undergoes continuous modifications.

These modifications make the software adaptable to the new business requirements and to make it interoperable with the modern computing environments.

Though, legacy software systems are becoming problem-in large organizations because of their high maintenance, they are still being used in the organizations due to the difficulties and risks encountered while replacing these systems in the modern systems. Legacy systems support essential business activities and therefore they are considered as business-critical systems.

Types of changes made to legacy systems

Re-engineering must be carried out on legacy systems so as to make these systems capable of handling the modern business requirements. This can be done by making the following significant changes to the legacy systems,

[Discover more](#)
[Implementing Enterprise AI And Machine Learning Tools](#)
[Software Requirements](#)
[Choosing Cloud Photo Sharing Platforms](#)

Making the software adaptable

The software can meet the requirements of new computing environments, by making the software adaptable to the computing environment.

Enhancing the software

The characteristic features of software need to be enhanced so that the software can easily implement the new business requirements.

Extending the software

The software needs to be extended so that it can achieve the interoperability feature.

Redesigning the software

The software needs to be redesigned by making changes to the existing software so as to make the software operable within a network environment.

Reasons for legacy software evolution

Software evolution is an evolutionary process where software is continuously changed over its lifetime in response to changing requirements. This process is carried by 'change' and is performed when,

- i) The errors identified are corrected.
- ii) The software becomes adaptable to new computing environment.
- iii) The application re-engineering is performed.

The following important laws are defined, so as to get a brief description about the unified theory for software evolution,

1. Continuing change law

The E-type system software that evolved over time must be continuously adapted so as to make them implement in real world computing environment. The system is capable of meeting the customers' requirements and satisfying them to the maximum extent only if the system undergoes a continuous modification.

2. Increasing complexity law

During the evolution process of an E-type system software, the level of complexity increases when no measures are used for reducing or maintaining it.

3. Self regulation law

The process of evolving an E-type system is self-regulating with respect to the product distribution and process measures. Its value is approximately equal to the normal value.

4. Conservation of organizational stability law

The average activity rate is constant over the lifetime of a product when an E-type system is being evolved.

5. Conservation of familiarity law

When an E-type system is being evolved, it is necessary to ensure that all the members (i.e., developers, sales personnel, users) responsible for performing the system evolution must maintain the entire information about the evolution process. They should also track the behavior using which a satisfactory evolution is achieved.

6. Continuing growth law

The functionality of an E-type system must increase continuously over the lifetime of the system in response to the customer's requirements.

7. Declining quality law

The quality of an E-type system will be degraded if the software system doesn't undergo continuous modification. Due to this, the system fails to meet the requirements of new computing environments.

8. Feedback system law

The evolution process of an E-type system comprises of feedback systems with multilevel, multiloop, multi-agent feature. It is necessary to have such feedback systems for improving the performance of the software

Software myths

Beliefs about software and the process used to build it can be traced to the earliest days of computing. Myths have a number of attributes that have made them insidious.

Management myths

Managers with software responsibility, like managers in most disciplines, are often under pressure to maintain budgets, keep schedules from slipping, and improve quality.

Myth: We already have a book that's full of standards and procedures for building software – won't that provide my people with everything they need to know?

Reality: The book of standards may very well exist but, is it used? Are software practitioners aware of its existence? Does it reflect modern software engineering practice?

Myth: If we get behind schedule, we can add more programmers and catch up.

Reality: Software development is not a mechanistic process like manufacturing. As new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.

[Discover more](#)
[Software engineering](#)
[Data Management](#)
[Project Management Software](#)

Myth: If I decide to outsource the software project to a third party, I can just relax and let that firm build it.

Reality: If an organization does not understand how to manage and control software projects internally, it will invariably struggle when it outsources software projects.

Customer myths

The customer believes myths about software because software managers and practitioners do little to correct misinformation. Myths lead to false expectations and ultimately, dissatisfaction with the developer.

Myth: A general statement of objectives is sufficient to begin writing programs – we can fill in the details later.

Reality: Although a comprehensive and stable statement of requirements is not always possible, an ambiguous statement of objectives is a recipe for disaster.

Myth: Project requirements continually change, but change can be easily accommodated because software is flexible.

Reality: It is true that software requirements change, but the impact of change varies with the time at which it is introduced. Change can cause upheaval that requires additional resources and major design modification.

Practitioner's myths

Myths that are still believed by software practitioners. During the early days of software, programming was viewed as an art. Old ways and attitudes die hard.

Myth: Once we write the program and get it to work, our jobs are done.

Reality: Someone once said that the sooner you begin writing code, the longer it'll take you to get done. Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.

[Discover more](#)
[programmers](#)
[Business & Productivity Software](#)
[Myth](#)

Myth: The only deliverable work product for a successful project is the working program.

Reality: A working program is only one part of a software configuration that includes many elements. Documentation provides guidance for software support.

Myth: Software engineering will make us create voluminous and unnecessary documentation and will invariably slow us down.

Reality: Software engineering is not about creating documents. It is about creating quality. Better quality leads to reduced rework. And reduced rework results in faster delivery times.

A Generic View of Process in Software Engineering:

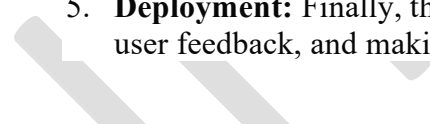
In software engineering, a **process** is essentially a structured set of activities, actions, and tasks required to build high-quality software. Think of it as a **roadmap or blueprint** that guides a software project from start to finish, ensuring that development is systematic, predictable, and efficient, rather than chaotic.

What is a Software Process?

A software process defines **how** software is created. It's a series of steps and practices that, when followed, aim to deliver a product that meets user needs, is reliable, and can be maintained over time. While many specific software development methodologies exist (like Waterfall, Agile, or DevOps), a “generic view” identifies the fundamental activities common to almost all of them.

Key Generic Process Activities

Regardless of the specific approach, a software project typically moves through these core, interconnected phases:

- 
1. **Communication:** This is about **understanding the problem**. It involves talking with customers and stakeholders to gather and clarify all the requirements for the software. What does the user want the software to do?
 2. **Planning:** Here, the team **defines the roadmap**. This involves estimating the time, cost, and resources needed, setting schedules, and identifying potential risks. It's about figuring out “how” the project will be completed within constraints.
 3. **Modeling:** This phase focuses on **designing the solution**. Software engineers create visual and conceptual representations (like blueprints or diagrams) of the software's structure, architecture, and how it will function. It's about translating requirements into a design that can be coded.
 4. **Construction:** This is where the **software is built**. It involves writing the actual code based on the design, followed by thorough testing to find and fix errors, ensuring the software works as intended and meets quality standards.
 5. **Deployment:** Finally, the software is **delivered to the customer**. This includes installing it, getting user feedback, and making it available for actual use.
- 

Why Have a Generic Process?

This generic view highlights that software development isn't just about coding. It's a disciplined engineering activity that requires **clear communication, careful planning, thoughtful design, robust construction, and effective delivery**. By following a defined process, software teams can manage complexity, improve quality, and increase the likelihood of project success.

Software engineering – A layered technology

Software engineering is a fully layered technology. To develop software, we need to go from one layer to another. All the layers are connected and each layer demands the fulfillment of the previous layer.



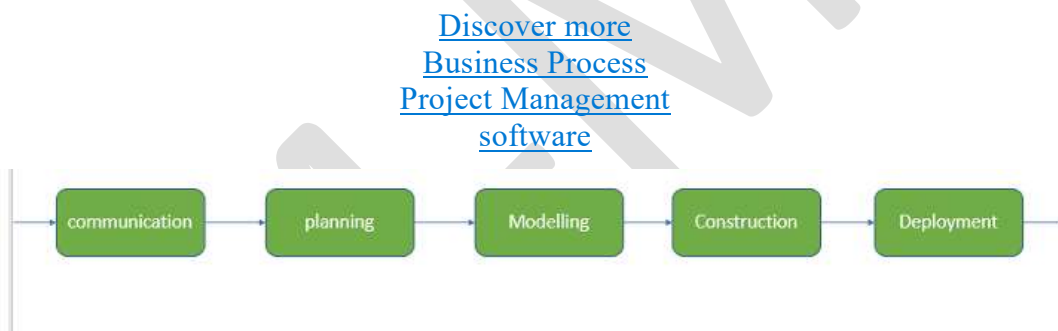
Layered technology is divided into four parts:

1. A quality focus

It defines the continuous process improvement principles of software. It provides integrity that means providing security to the software so that data can be accessed by only an authorized person, no outsider can access the data. It also focuses on maintainability and usability.

2. Process

It is the foundation or base layer of software engineering. It is key that binds all the layers together which enables the development of software before the deadline or on time. Process defines a framework that must be established for the effective delivery of software engineering technology. The software process covers all the activities, actions, and tasks required to be carried out for software development.



3. Method

During the process of software development the answers to all “how-to-do” questions are given by method. It has the information of all the tasks which includes communication, requirement analysis, design modeling, program construction, testing, and support.

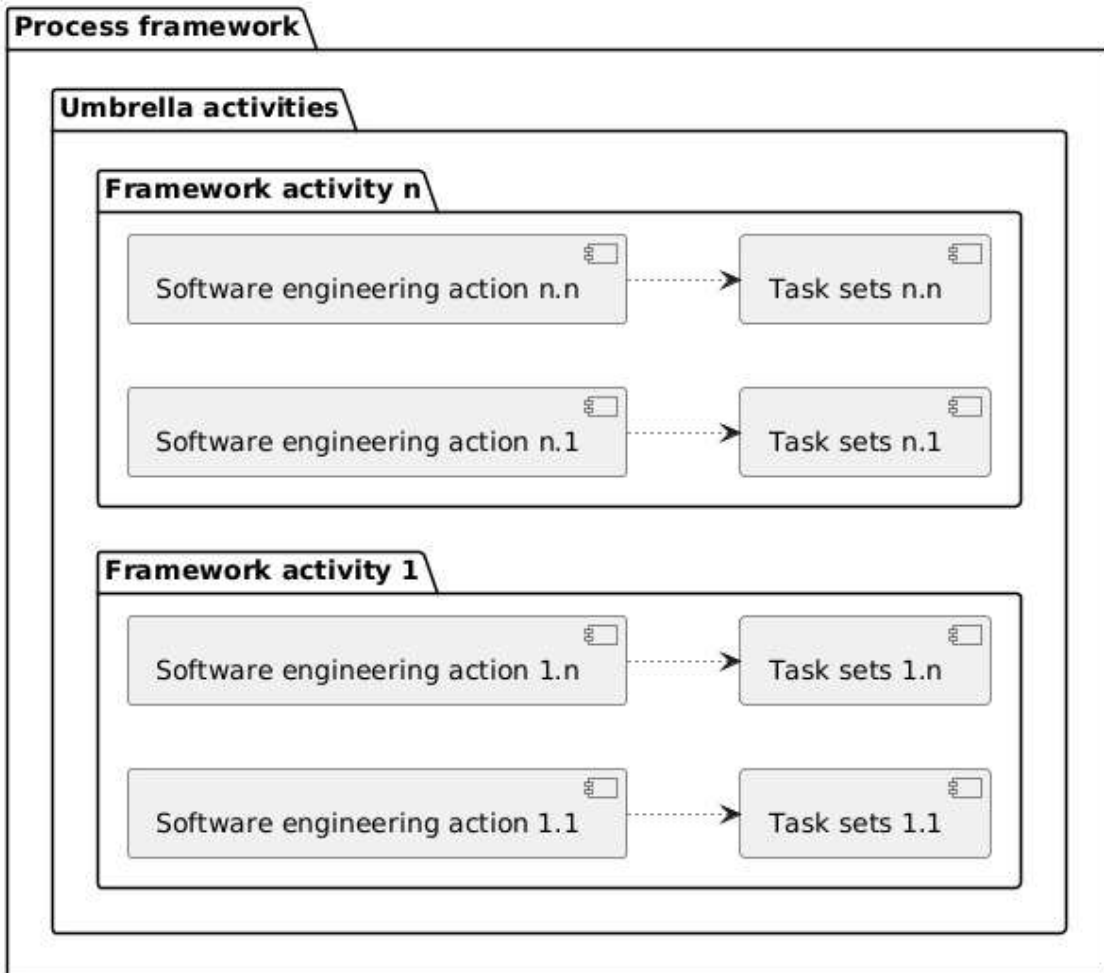
4. Tools

Software engineering tools provide a self-operating system for processes and methods. Tools are integrated which means information created by one tool can be used by another.

A process framework

Process framework plays a major role in every software development activity as it forms a base in initiating the development process. It is used to estimate certain framework activities which are applied in very developmental activity, irrespective of its size. The process framework includes several umbrella activities, which are useful throughout the software development process. Next to umbrella activity is a framework activity which includes definite number of software engineering actions applicable in driving a specific software engineering application. Each of these engineering actions corresponds to the task set which encompasses a list of actions to be applied. Each engineering action can perform a constituent activity which forms an essential part of the specific software application being developed.

The five generic process framework activities useful in developing several projects are given below:



1. Communication

This refers to a framework activity where, usually the end users (say customers) are communicated and their views related to the project are analyzed. Here, reports related to customer requirement specifications are developed.

[Discover more projects](#)
[Software engineering Programming](#)

2. Planning

In this framework activity, usually the entire work schedule which is going to be implemented in further stages is prepared. Hence, various issues to be addressed during this framework are risks, requirement of resources, software schedules, important products to be developed etc.

3. Modelling

Once the developer is done with analyzing the customer requirement specifications, the third framework activity will be modeling. Here, usually UML diagrams are used to represent the project in the form of architecture. This helps developers and the customer to gain an insight of the end product.

4. Construction

It is a combination of code generation and testing.

5. Deployment

In this framework activity, usually the developed project is delivered to the end users. They deploy the project and give relative feedbacks to the developers.

Umbrella activities

There are eight umbrella activities of a software process framework. They are:

- Tracking and controlling software project
- Managing risks
- Software Quality Assurance (SQA)
- Formal technical reviews
- Software measurements
- Software Configuration Management (SCM)
- Managing reusability factor
- Preparing and producing software work products

Tracking and controlling software project

The first umbrella activity is tracking and controlling the software project. This activity allows the software team to perform the following tasks:

[Discover more](#)

[Project Management Software](#)

[Engineering & Technology](#)

[project](#)

- Assessing of progress of the project against the plan of the project
- Maintaining the schedule of the project by taking appropriate action based on above assessment

Managing risks

This activity involves the following tasks:

- Evaluation of those risks that can have a serious impact on the final result of the project
- Assessment of those risks that are likely to affect the product's quality

Software Quality Assurance (SQA)

This activity ensures the software quality by defining and organizing the activities needed to assure quality of software.

Formal technical reviews

This activity tries to eliminate errors as quickly as possible so that they don't affect the other activities. It is done by evaluation of work products of software engineering.

Software measurements

This activity describes as well as gathers measures of process, product, and project through which software team can deliver a software that fulfills the needs of the customer.

Software Configuration Management (SCM)

SCM is also referred to as Change Management (CM). During the software engineering process, SCM defines a set of activities for managing the changes made to the software components.

Managing reusability factor

This activity is incorporated by the software process framework because of the following reasons:

- It defines the basic criteria for reusing a work product. Other than this, it also defines a criteria for software components reuse
- It also obtains reusable components by developing the desired methods

Preparing and producing software work products

The last and final activity is the preparation and production of work products. It includes certain activities that are needed for the creation of the following work products:

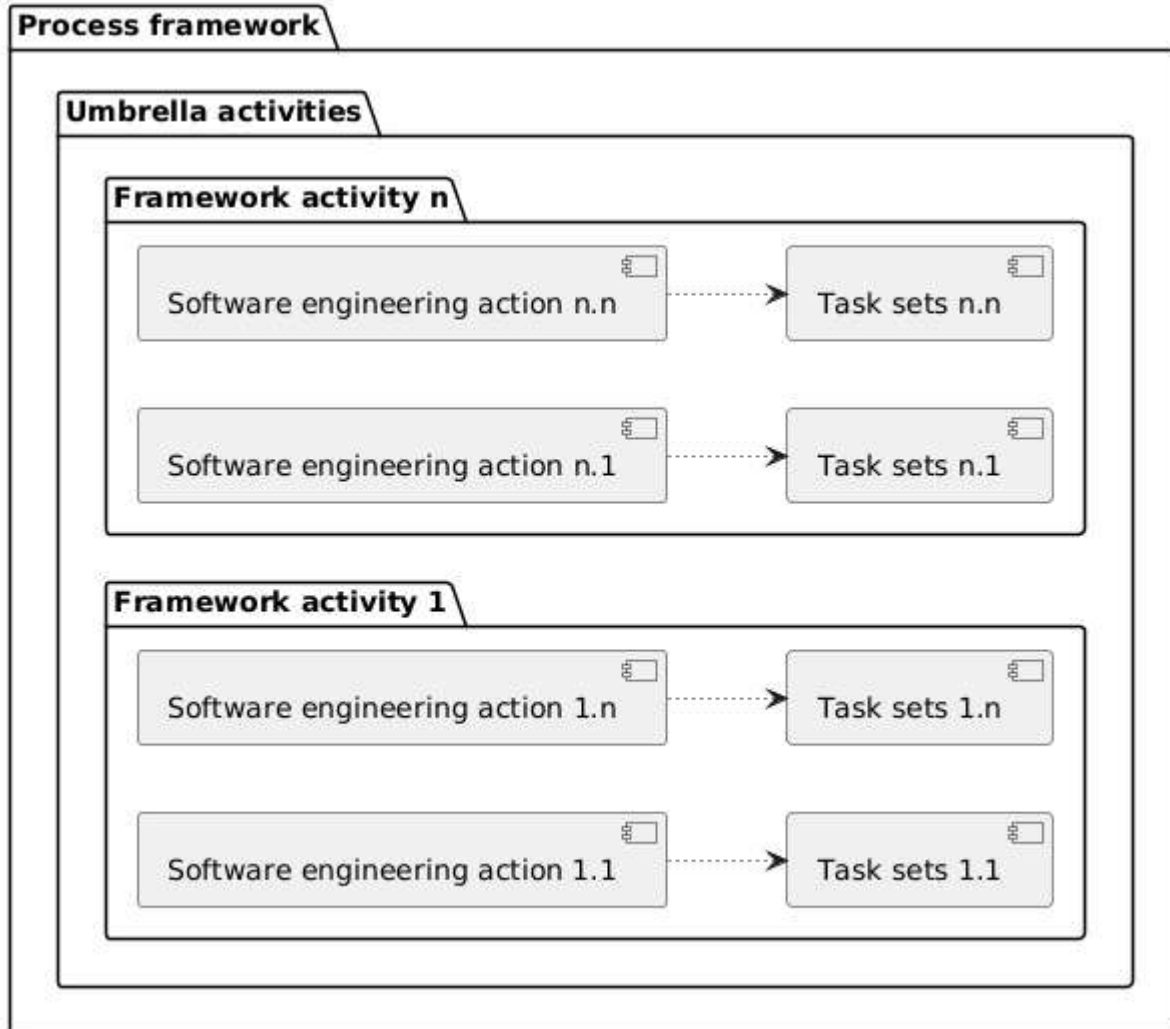
- Models
- Logs
- Documents
- Lists and forms

A process framework

Process framework plays a major role in every software development activity as it forms a base in initiating

the development process. It is used to estimate certain framework activities which are applied in very developmental activity, irrespective of its size. The process framework includes several umbrella activities, which are useful throughout the software development process. Next to umbrella activity is a framework activity which includes definite number of software engineering actions applicable in driving a specific software engineering application. Each of these engineering actions corresponds to the task set which encompasses a list of actions to be applied. Each engineering action can perform a constituent activity which forms an essential part of the specific software application being developed.

The five generic process framework activities useful in developing several projects are given below:



1. Communication

This refers to a framework activity where, usually the end users (say customers) are communicated and their views related to the project are analyzed. Here, reports related to customer requirement specifications are developed.

2. Planning

In this framework activity, usually the entire work schedule which is going to be implemented in further stages is prepared. Hence, various issues to be addressed during this framework are risks, requirement of resources, software schedules, important products to be developed etc.

[Discover more
Programming
software engineering
Operating Systems](#)

3. Modelling

Once the developer is done with analyzing the customer requirement specifications, the third framework activity will be modeling. Here, usually UML diagrams are used to represent the project in the form of architecture. This helps developers and the customer to gain an insight of the end product.

4. Construction

It is a combination of code generation and testing.

5. Deployment

In this framework activity, usually the developed project is delivered to the end users. They deploy the project and give relative feedbacks to the developers.

Umbrella activities

There are eight umbrella activities of a software process framework. They are:

- Tracking and controlling software project
- Managing risks
- Software Quality Assurance (SQA)
- Formal technical reviews
- Software measurements
- Software Configuration Management (SCM)
- Managing reusability factor
- Preparing and producing software work products

Tracking and controlling software project

The first umbrella activity is tracking and controlling the software project. This activity allows the software team to perform the following tasks:

- Assessing of progress of the project against the plan of the project
- Maintaining the schedule of the project by taking appropriate action based on above assessment

Managing risks

This activity involves the following tasks:

[Discover more](#)

[Machine Learning & Artificial Intelligence](#)

[Business & Corporate Law](#)

[Data](#)

- Evaluation of those risks that can have a serious impact on the final result of the project
- Assessment of those risks that are likely to affect the product's quality

Software Quality Assurance (SQA)

This activity ensures the software quality by defining and organizing the activities needed to assure quality of software.

Formal technical reviews

This activity tries to eliminate errors as quickly as possible so that they don't affect the other activities. It is done by evaluation of work products of software engineering.

Software measurements

This activity describes as well as gathers measures of process, product, and project through which software team can deliver a software that fulfills the needs of the customer.

Software Configuration Management (SCM)

SCM is also referred to as Change Management (CM). During the software engineering process, SCM defines a set of activities for managing the changes made to the software components.

Managing reusability factor

This activity is incorporated by the software process framework because of the following reasons:

- It defines the basic criteria for reusing a work product. Other than this, it also defines a criteria for software components reuse
- It also obtains reusable components by developing the desired methods

Preparing and producing software work products

The last and final activity is the preparation and production of work products. It includes certain activities that are needed for the creation of the following work products:

[Discover more](#)

[Ethics](#)

[Intellectual Property](#)

[Privacy Issues](#)

- Models
- Logs
- Documents
- Lists and forms

The capability maturity model integration (CMMI)

Capability Maturity Model Integration (CMMI) is a successor of CMM and is a more evolved model that incorporates best components of individual disciplines of CMM like Software CMM, Systems Engineering CMM, People CMM, etc. Since CMM is a reference model of matured practices in a specific discipline, it becomes difficult to integrate these disciplines as per the requirements. This is why CMMI is used as it allows the integration of multiple disciplines as and when needed.

Objectives of CMMI

1. Fulfilling customer needs and expectations
2. Value creation for investors/stockholders
3. Market growth is increased
4. Improved quality of products and services
5. Enhanced reputation in industry

CMMI representation – Staged and continuous

A representation allows an organization to pursue a different set of improvement objectives. There are two representations for CMMI:

Staged representation

- Uses a pre-defined set of process areas to define improvement path
- Provides a sequence of improvements, where each part in the sequence serves as a foundation for the next
- An improved path is defined by maturity level
- Maturity level describes the maturity of processes in organization
- Staged CMMI representation allows comparison between different organizations for multiple maturity levels

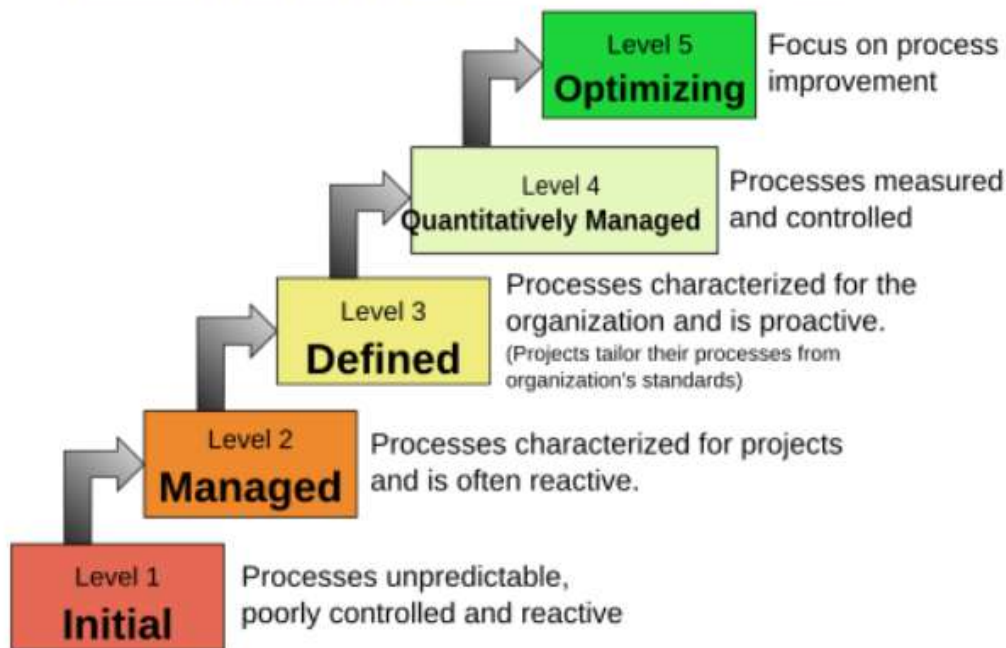
Continuous representation

- Allows selection of specific process areas
- Uses capability levels that measure improvement of an individual process area
- Continuous CMMI representation allows comparison between different organizations on a process-area-by-process-area basis
- Allows organizations to select processes which require more improvement
- In this representation, order of improvement of various processes can be selected which allows the organizations to meet their objectives and eliminate risks

CMMI model – Maturity levels

In CMMI with staged representation, there are five maturity levels described as follows:

Characteristics of the Maturity levels



Maturity level 1: Initial

- Processes are poorly managed or controlled
- Unpredictable outcomes of processes involved
- Ad hoc and chaotic approach used
- No KPAs (Key Process Areas) defined
- Lowest quality and highest risk

Maturity level 2: Managed

- Requirements are managed
- Processes are planned and controlled
- Projects are managed and implemented according to their documented plans
- The risk involved is lower than Initial level, but still exists
- Quality is better than Initial level

Maturity level 3: Defined

- Processes are well characterized and described using standards, proper procedures, and methods, tools, etc.
- Medium quality and medium risk involved
- Focus is process standardization

Maturity level 4: Quantitatively managed

- Quantitative objectives for process performance and quality are set
- Quantitative objectives are based on customer requirements, organization needs, etc.
- Process performance measures are analyzed quantitatively
- Higher quality of processes is achieved
- Lower risk

Maturity level 5: Optimizing

- Continuous improvement in processes and their performance
- Improvement has to be both incremental and innovative
- Highest quality of processes
- Lowest risk in processes and their performance

CMMI model – Capability levels

A capability level includes relevant specific and generic practices for a specific process area that can improve the organization's processes associated with that process area. For CMMI models with continuous representation, there are six capability levels as described:

[Discover more](#)
[Career Resources & Planning](#)
[Charity & Philanthropy](#)
[Engineering & Technology](#)

Capability level 0: Incomplete

- Incomplete process – partially or not performed
- One or more specific goals of process area are not met
- No generic goals are specified for this level
- This capability level is same as maturity level 1

Capability level 1: Performed

- Process performance may not be stable
- Objectives of quality, cost and schedule may not be met
- A capability level 1 process is expected to perform all specific and generic practices for this level
- Only a start-step for process improvement

Capability level 2: Managed

- Process is planned, monitored and controlled
- Managing the process by ensuring that objectives are achieved
- Objectives are both model and other including cost, quality, schedule
- Actively managing processing with the help of metrics

Capability level 3: Defined

- A defined process is managed and meets the organization's set of guidelines and standards
- Focus is process standardization

Capability level 4: Quantitatively managed

- Process is controlled using statistical and quantitative techniques
- Process performance and quality is understood in statistical terms and metrics
- Quantitative objectives for process quality and performance are established

Capability level 5: Optimizing

- Focuses on continually improving process performance
- Performance is improved in both ways – incremental and innovation
- Emphasizes on studying the performance results across the organization to ensure that common causes or issues are identified and fixed

Process patterns

The software process can be defined as a collection of patterns that define a set of activities, actions, work tasks, work products, and/or related behaviors required to develop computer software.

A process pattern provides us with a template—a consistent method for describing an important characteristic of the software process. A pattern might be used to describe a complete process or a task within a framework activity.

Pattern name

The pattern is given a meaningful name that describes its function within the software process.

Intent

The objective of the pattern is described briefly.

[Discover more](#)
[Dictionaries & Encyclopedias](#)

Type

The pattern type is specified. There are three types:

Task patterns

- Define a software engineering action or work task that is part of the process
- Relevant to successful software engineering practice
- **Example:** Requirement Gathering

Stage patterns

- Define a framework activity for the process
- Incorporate multiple task patterns that are relevant to the stage
- **Example:** Communication

Phase patterns

- Define the sequence of framework activities that occur within the process
- Apply even when the overall flow of activities is iterative in nature
- **Example:** Spiral model or Prototyping

Initial context

The conditions under which the pattern applies are described before the pattern begins. We ask:

[Discover more](#)
[software engineering](#)
[Civil Engineering](#)
[Visual Art & Design](#)

- What organizational or team-related activities have already occurred?
- What is the entry state for the process?
- What software engineering or project information already exists?

Problem

The problem to be solved by the pattern is clearly described.

Solution

The implementation of the pattern is explained.

This section describes:

- How the initial state of the process is modified after the pattern begins
- How the available project or software engineering information is transformed after the pattern is successfully executed

Resulting context

The conditions that will result once the pattern has been successfully implemented are described. Upon completion, we ask:

- What organizational or team-related activities must have occurred?
- What is the exit state for the process?
- What software engineering or project information has been developed?

Known uses

The specific instances where the pattern is applicable are indicated.

[Discover more](#)
[Project Management Software](#)
[engineering](#)
[Graphic Design](#)

Process patterns provide an effective mechanism for describing any software process.

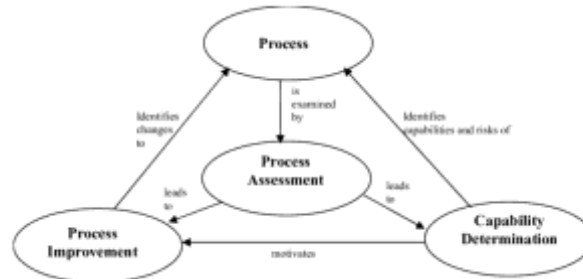
These patterns enable a software engineering organization to develop a hierarchical process description

that begins at a high level of abstraction.

Once process patterns have been developed, they can be reused to define process variants. A customized process model can then be defined by a software team using these patterns as building blocks.

Process assessment

The existence of a software process does not guarantee that the software will be delivered on time, that it will meet the customer's needs, or that it will demonstrate the long-term quality characteristics required. Therefore, the process itself must be assessed to ensure it satisfies essential criteria for successful software



engineering.

Several approaches to software process assessment have been developed over the past few decades.

SCAMPI (Standard CMMI Assessment Method for Process Improvement)

- Provides a **five-step** process assessment model.
- The five steps are:
 - Initiating
 - Diagnosing
 - Establishing
 - Acting
 - Learning
- Uses the **SEI CMMI** as the basis for assessment.

CBA IPI (CMM-Based Appraisal for Internal Process Improvement)

- A diagnostic technique for assessing the **relative maturity** of a software organization.
- Uses the **SEI CMM** as the basis for evaluation.

SPICE (Software Process Improvement and Capability Determination)

- Defined by **ISO/IEC 15504** standard.
- Provides a set of requirements for software process assessments.
- The main goal is to help organizations **objectively evaluate** the effectiveness of their software process.

ISO 9001:2000 for Software

- A **generic standard** applicable to any organization aiming to improve quality.
- Directly relevant to **software organizations and companies**.
- Focuses on improving overall quality of **products, systems, or services**.

Personal and team process models

The best software process is one that is closely aligned with the people performing the work. Each software engineer should create a process that suits their individual needs while also supporting the goals of the team and the organization. Alternatively, the team can create a shared process that balances individual and organizational objectives.

Personal Software Process (PSP)

The **Personal Software Process (PSP)** focuses on the individual engineer, emphasizing measurement of both the work product and its quality.

Framework activities of PSP

1. Planning

- Requirements are identified.
- Size and resource estimates are developed.
- A defect estimate is created.
- All data is recorded using worksheets or templates.
- Development tasks are listed, and a project schedule is prepared.

2. High-level design

- External specifications for components are developed.
- Component designs are created.
- Prototypes are built to resolve uncertainties.
- Issues are recorded and tracked.

3. High-level design review

- Formal verification techniques are used to detect design errors.
- Metrics are recorded for all important tasks and outputs.

4. Development

- Component-level designs are refined and reviewed.
- Code is written, reviewed, compiled, and tested.
- Metrics are maintained throughout.

5. Postmortem

- Collected metrics are analyzed to evaluate process effectiveness.
- Metrics guide changes to improve future process performance.

Key points

- PSP encourages engineers to **identify and understand their own errors early**.
- It provides a **disciplined, metrics-based** approach to software engineering.

Team Software Process (TSP)

The **Team Software Process (TSP)** aims to build a **self-directed project team** that is organized and accountable for producing high-quality software.

Objectives of TSP

- Create **self-directed teams** that manage their own planning, tracking, goal-setting, and process ownership.
- Teams can be software-only or integrated product teams (IPTs) of 3 to 20 engineers.
- Teach managers how to **coach, motivate**, and sustain high team performance.
- Speed up software process improvement by adopting behaviors aligned with **CMM Level 5**.
- Offer improvement guidance to **high-maturity organizations**.
- Support university instruction in **industrial-grade team skills**.

Characteristics of a TSP team

- Defines roles and responsibilities for every member.
- Tracks **quantitative project data**.
- Establishes a team-specific software process.
- Develops a strategy to implement the process.
- Defines local standards for software engineering work.
- Continuously assesses and responds to project risks.
- **Monitors and reports** project status.

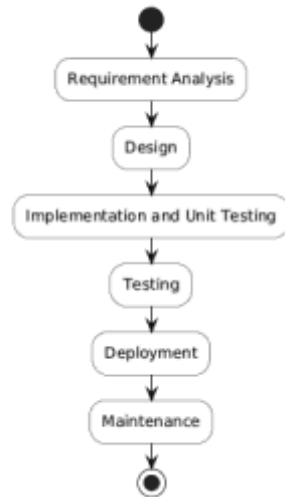
Process models

Software development life cycle

Software development life cycle (SDLC) is a structured process that is used to design, develop, and test good-quality software. SDLC, or software development life cycle, is a methodology that defines the entire procedure of software development step-by-step. The goal of the SDLC life cycle model is to deliver high-quality, maintainable software that meets the user's requirements. SDLC in software engineering models outlines the plan for each stage so that each stage of the software development model can perform its task efficiently to deliver the software at a low cost within a given time frame that meets user's requirements.

The SDLC Model involves six phases or stages while developing any software.

Waterfall Model - Software Development Phases



Requirements analysis phase

Requirements analysis and specification phase is very important in the successful development of a software. If the customer requirements are not analyzed properly, the chances of project failure or amount of rework on project becomes very high. Therefore, the project startup without proper requirements analysis and documentation is considered as a major mistake.

It also increases the burden on developer to reflect/implement the changes multiple times. Such iterations increase the cost incurred in the overall project and also leads to customer dissatisfaction.

The importance of requirements understanding and documentation is independent of the type of project i.e., internal or contract based. The documentation generated after successful analysis and specification is called Software Requirement Specification (SRS) document. This document not only helps in getting a clear understanding about product requirements, but also helps in carrying out different activities of later phases. Due to this fact, expert developers spend a lot of time to gather requirements.

[Discover more](#)
[Business Process](#)
[Development Life Cycle](#)
[Software development](#)

Design phase

Design is a meaningful engineering representation of something that is to be built. It can be traced to a customer's requirements and at the same time assessed for quality against a set of predefined criteria for "good" design. In the software engineering context, design focuses on four major areas of concern – data, architecture, interfaces and components.

Software design process is an iterative process through which requirements document (SRS) is transformed to design document.

Implementation and unit testing phase

In this phase, actual code is developed using a programming language. As soon as the code is developed, each unit or component in it is tested individually. Such testing is called unit testing. It ensures that maximum errors are detected and entire code is tested by following certain paths in the control structure.

Testing phase

Testing is a major activity for software development and its main function is to detect bugs or errors that occur while executing a software. During the phases like requirement analysis and system design, the output results in the form of textual documents whereas, for the coding phase, the output is a computer program. Hence, testing not only detect errors during

[Discover more](#)

[Data](#)

[Financial Markets News](#)

[Intellectual Property](#)

Development Life Cycle coding but also detect errors that occur during the previous development phases. Subsequently, there are four different types of testing that can be performed on software development.

Deployment phase

This forms the final phase of the software development process where the software is usually delivered to the end users for its effective implementation. Later, feedback is collected from the users and if, the software fails to meet the users requirements, it is modified. Hence, the important activities involved in this phase are:-

- Perfect delivery
- Support and
- Feedbacks.

Maintenance phase

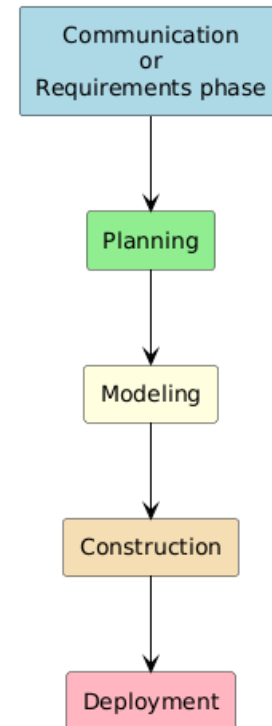
Software maintenance refers to the changes which are performed once the software is delivered to users. Maintenance activity is compulsory for each type of product and cannot be avoided. Generally, products require maintenance activity due to its heavy usage. But, software need maintenance in order to improve features, correct errors. It is an essential activity adopted by large number of organizations because of the misuse of hardware.

Waterfall model

Introduction

Waterfall model remains one of the oldest strategies ever applied, in the development of a software. It is also known as classic life cycle model, which divides the entire software development process into five main phases. Following is the diagrammatical representation of waterfall model,

The waterfall model was proposed with feedback loops. However, most of the organizations avoid these



loops. Thus this model is also called Linear Sequential Model.

1. Communication or requirements phase

In the first phase, the customer requirements are retrieved to generate customer requirements specification. Hence, all the stakeholders (or the end users) are called and their vision about the end product is enquired/collected and documented.

There are two important facts about this phase:

- Project initiation
- Requirements gathering

2. Analysis or planning phase

In this phase, the entire project strategy is devised. The main addressable issues in this phase are as follows:

[Discover more](#)
[Business Process](#)
[Customer Services](#)
[Project Management](#)

- The entire project schedule is estimated.
- A thorough analysis of resource requirements is made.
- Number of people involved in the project is determined.
- Duration and expected cost of the project are estimated

Therefore, the main focus of this phase remains on:

- Estimation
- Scheduling
- Tracking

3. Modeling phase

In this phase, the entire project scenario which was analyzed in the second phase is modelled diagrammatically. To do so, the UML diagrams are used. Modeling remains important for successful completion of the project.

The two important aspects of this phase are:

- Analysis
- Design

4. Construction phase

This is usually the coding phase of the software. Here, each component of the software is coded and is suitably integrated. Once the coding part is completed, the entire software is thoroughly tested.

[Discover more](#)

[Customer Relationship Management \(CRM\)](#)

[Project Management Software](#)

[Projects](#)

Hence, the two most important activities performed during this phase are:

- Coding
- Testing

5. Deployment phase

This is the final phase in which the software is usually delivered to the end users for its effective implementation. Later, feedback is collected from the users. If the software fails to meet the users requirements, it is modified.

Hence, the important activities involved in this phase are:

- Perfect delivery
- Support
- Feedbacks

Limitations of the model

Though the above software development process looked quite straight forward, it is now rarely used. Even the organizations, which remained loyal to this strategy, started raising serious objections, which added to its failure.

Advantages

1. Simple and Easy to Understand: The Waterfall model is straightforward, making it easy to understand and implement, especially for smaller projects or teams without complex requirements.
2. Clear Structure and Documentation: Since each phase is well-defined, it's easier to document and track progress. This helps with maintaining clear requirements and specifications throughout the project lifecycle.
3. Early Planning: The project scope and requirements are clearly defined at the beginning, which can help prevent scope creep and give stakeholders clear expectations about timelines and costs.
4. Easy to Manage: Due to its linear structure, the Waterfall model is relatively simple to manage, particularly for projects where requirements are well understood upfront.
5. Well-Suited for Small Projects: For smaller projects with fixed requirements and minimal changes, Waterfall is a good fit because it minimizes the need for iteration.

Disadvantages

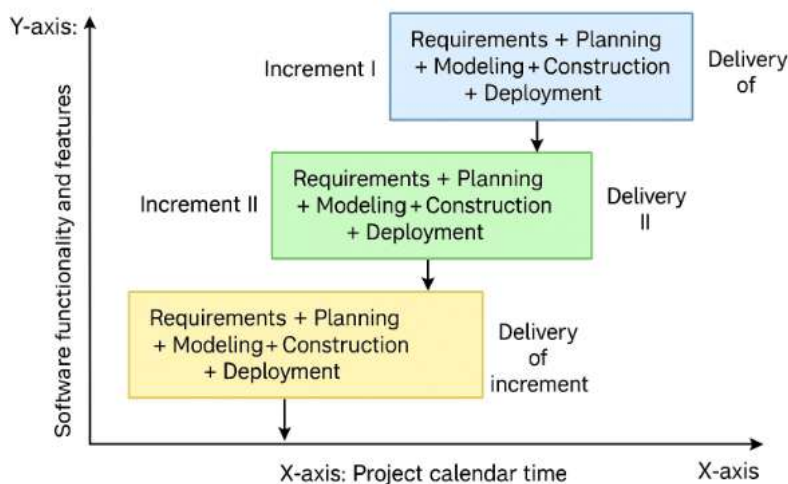
1. Inflexible to Changes: Once a phase is completed, going back and making changes can be difficult and costly. The model assumes all requirements can be fully defined at the beginning, which doesn't always align with real-world scenarios where needs evolve.

2. Late Testing: Testing only happens after the implementation phase, which can lead to discovering critical issues late in the development process. This increases the risk of rework and delays.
3. Assumes All Requirements are Known: Waterfall works best when requirements are fully known from the start. However, in many projects, requirements evolve or are unclear at the outset, which can make it hard to apply the Waterfall model effectively.
4. Limited Customer Involvement: Since the Waterfall model focuses on completing each phase before moving to the next, there is limited customer feedback during development. This means any misalignment with customer needs might not be identified until the later stages.
5. Not Ideal for Complex Projects: For large, complex, or high-risk projects, Waterfall can be too rigid and inefficient, as it doesn't allow for flexibility and iterative learning that might be needed to handle complex requirements.

Incremental model and RAD process

Incremental model

The incremental model is a software development approach where the system is designed, implemented, and tested incrementally (in small portions or “increments”). This model allows the system to be developed in parts, with each increment adding more functionality over time. It's commonly used in situations where requirements are expected to evolve or when the development process needs to be



flexible.

When we use the incremental model?

- When the requirements are superior.
- A project has a lengthy development schedule.
- When software team are not very well skilled or trained.
- When the customer demands a quick release of the product.
- You can develop prioritized requirements first.

Advantage of incremental model

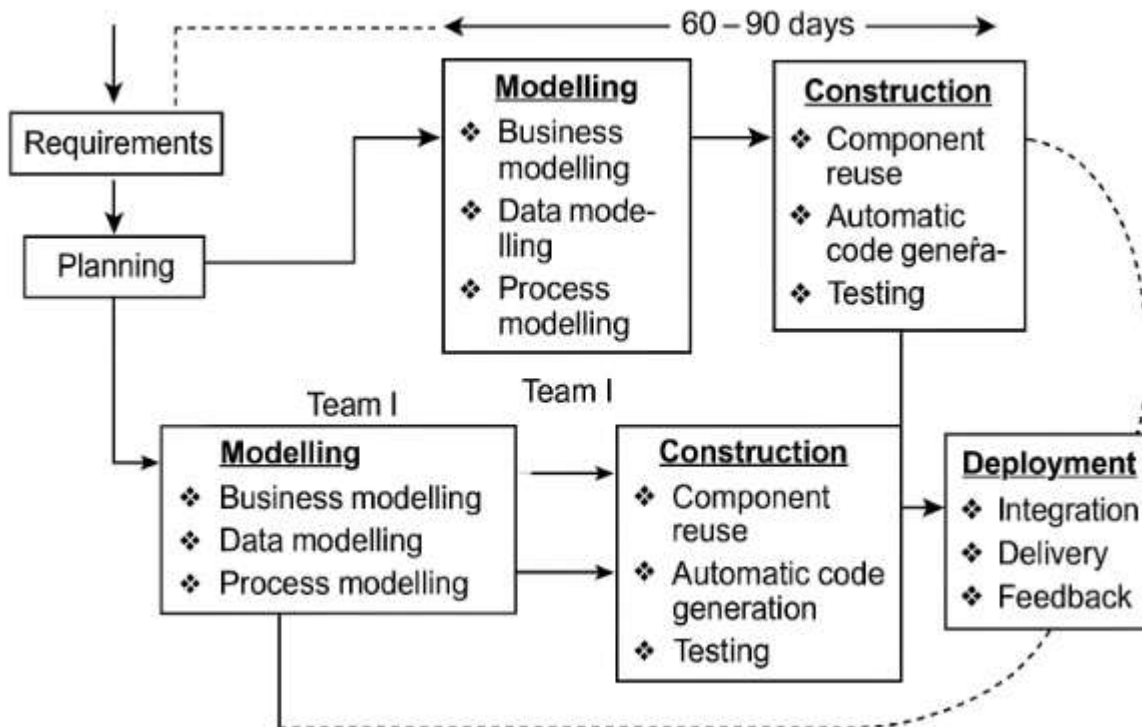
- Errors are easy to be recognized.
- Easier to test and debug
- More flexible.
- Simple to manage risk because it handled during its iteration.
- The client gets important functionality early.

Disadvantage of incremental model

- Need for good planning
- Total cost is high.
- Well defined module interfaces are needed.

RAD process

The RAD model or Rapid Application Development model is a type of software development methodology that emphasizes quick and iterative release cycles, primarily focusing on delivering working software in shorter timelines. Unlike traditional models such as the Waterfall model, RAD is designed to be more flexible and responsive to user feedback and changing requirements throughout the development



process.

When to use the RAD model?

1. Well-understood requirements: When project requirements are stable and transparent, RAD is appropriate.
2. Time-sensitive projects: Suitable for projects that need to be developed and delivered quickly due to tight deadlines.
3. Small to medium-sized projects: Better suited for smaller initiatives requiring a controllable number of team members.
4. High user involvement: Fits where ongoing input and interaction from users are essential.
5. Innovation and creativity: Helpful for tasks requiring creative inquiry and innovation.
6. Prototyping: It is necessary when developing and improving prototypes is a key component of the development process.
7. Low technological complexity: Suitable for tasks using comparatively straightforward technological specifications.

Advantages of Rapid Application Development model (RAD)

- The use of reusable components helps to reduce the cycle time of the project.
- Feedback from the customer is available at the initial stages.
- Reduced costs as fewer developers are required.
- The use of powerful development tools results in better quality products in comparatively shorter periods.
- The progress and development of the project can be measured through the various stages.

Disadvantages of Rapid Application Development model (RAD)

- The use of powerful and efficient tools requires highly skilled professionals.
- The absence of reusable components can lead to the failure of the project.
- The team leader must work closely with the developers and customers to close the project on time.
- The systems which cannot be modularized suitably cannot use this model.
- Customer involvement is required throughout the life cycle.

Evolutionary, prototyping and spiral models

The evolutionary model

The evolutionary model is a combination of the Iterative and Incremental models of the software development life cycle. Delivering your system in a big bang release, delivering it in incremental process over time is the action done in this model. Some initial requirements and architecture envisioning need to be done. It is better for software products that have their feature sets redefined during development because of user feedback and other factors.

Application of evolutionary model

1. It is used in large projects where you can easily find modules for incremental implementation. Evolutionary model is commonly used when the customer wants to start using the core features instead of waiting for the full software.
2. Evolutionary model is also used in object oriented software development because the system can be easily portioned into units in terms of objects.

Necessary conditions for implementing this model

1. Customer needs are clear and been explained in deep to the developer team.
2. There might be small changes required in separate parts but not a major change.
3. As it requires time, so there must be some time left for the market constraints.
4. Risk is high and continuous targets to achieve and report to customer repeatedly.
5. It is used when working on a technology is new and requires time to learn.

Advantages evolutionary model

1. Adaptability to changing requirements: Evolutionary models work effectively in projects when the requirements are ambiguous or change often. They support adjustments and flexibility along the course of development.
2. Early and gradual distribution: Functional components or prototypes can be delivered early thanks to incremental development. Faster user satisfaction and feedback may result from this.
3. User commentary and involvement: Evolutionary models place a strong emphasis on ongoing user input and participation. This guarantees that the software offered closely matches the needs and expectations of the user.
4. Improved handling of difficult projects: Big, complex tasks can be effectively managed with the help of evolutionary models. The development process is made simpler by segmenting the project into smaller, easier-to-manage portions.

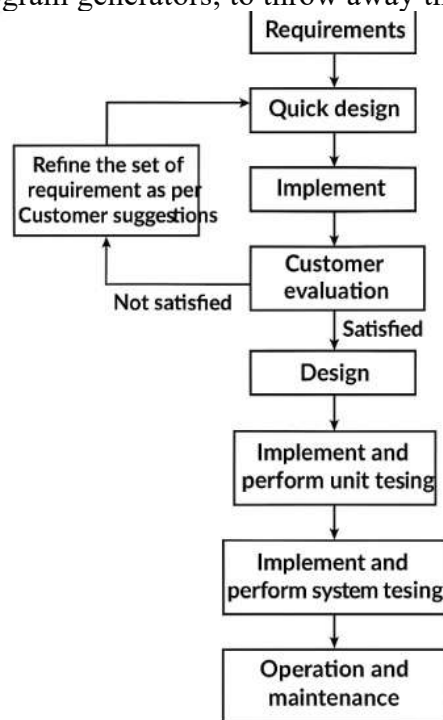
Disadvantages evolutionary model

1. Communication difficulties: Evolutionary models require constant cooperation and communication. The strategy may be less effective if there are gaps in communication or if team members are spread out geographically.
2. Dependence on an expert group: A knowledgeable and experienced group that can quickly adjust to changes is needed for evolutionary models. Teams lacking experience may find it difficult to handle these model's dynamic nature.
3. Increasing management complexity: Complexity can be introduced by organizing and managing several increments or iterations, particularly in large projects. In order to guarantee integration and synchronization, good project management is needed.
4. Greater initial expenditure: As evolutionary models necessitate continual testing, user feedback and prototyping, they may come with a greater starting cost. This may be a problem for projects that have limited funding.



Prototype

In prototyping model initially the requirement gathering is done Developer and customer defines: overall objectives, identify areas needing, more requirement gathering then a quick design is prepared this design represents what will be visible to user -in input and output format → when working prototype is built, developer use existing program fragments or program generators, to throw away the prototype, and



rebuild the system to high quality.

Steps of prototyping model

Step 1: Requirement gathering

This is the initial step in designing a prototype model. In this phase, users are asked about what they expect or what they want from the system.

Step 2: Quick design

This is the second step in the Prototyping Model. This model covers the basic design of the requirement through which a quick overview can be easily described.

Step 3: Implement

This step helps in building an actual prototype from the knowledge gained from prototype design.

Step 4: Customer evaluation

This step describes the preliminary testing where the investigation of the performance model occurs, as the customer will tell the strengths and weaknesses of the design, which was sent to the developer.

[Discover more](#)

[Biological Sciences](#)

[Development Tools](#)

[Design](#)

Step 5: Refining prototype

If any feedback is given by the user, then improving the client's response to feedback and suggestions, the final system is approved.

Step 6: Implement product and maintain

This is the final step in the phase of the Prototyping Model where the final system is tested and distributed to production, here the program is run regularly to prevent failures.

Advantages of prototyping model

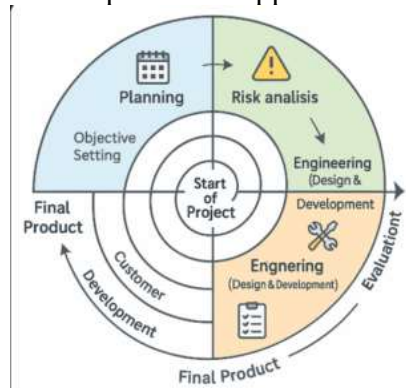
- The customers get to see the partial product early in the life cycle. This ensures a greater level of customer satisfaction and comfort.
- New requirements can be easily accommodated as there is scope for refinement.
- Missing functionalities can be easily figured out.
- Errors can be detected much earlier thereby saving a lot of effort and cost, besides enhancing the quality of the software.
- The developed prototype can be reused by the developer for more complicated projects in the future.
- Flexibility in design.
- Early feedback from customers and stakeholders can help guide the development process and ensure that the final product meets their needs and expectations.

Disadvantages of the prototyping model

- Costly concerning time as well as money.
- There may be too much variation in requirements each time the prototype is evaluated by the customer.
- Poor documentation due to continuously changing customer requirements.
- It is very difficult for developers to accommodate all the changes demanded by the customer.
- There is uncertainty in determining the number of iterations that would be required before the prototype is finally accepted by the customer.
- After seeing an early prototype, the customers sometimes demand the actual product to be delivered soon.
- Developers in a hurry to build prototypes may end up with sub-optimal solutions.

The spiral model

The Spiral Model is one of the most important Software Development Life Cycle models. The Spiral Model is a combination of the waterfall model and the iterative model. It provides support for risk



handling. The Spiral Model was first proposed by Barry Boehm.

Each phase of the spiral model is divided into four quadrants

1. Objectives determination and identify alternative solutions

Requirements are gathered from the customers and the objectives are identified, elaborated, and analyzed at the start of every phase. Then alternative solutions possible for the phase are proposed in this quadrant.

[Discover more Programming projects design](#)

2. Identify and resolve risks

During the second quadrant, all the possible solutions are evaluated to select the best possible solution. Then the risks associated with that solution are identified and the risks are resolved using the best possible strategy. At the end of this quadrant, the prototype is built for the best possible solution.

3. Develop the next version of the product

During the third quadrant, the identified features are developed and verified through testing. At the end of the third quadrant, the next version of the software is available.

4. Review and plan for the next phase

In the fourth quadrant, the customers evaluate the so-far developed version of the software. In the end, planning for the next phase is started.

Advantages of the spiral model

1. Risk handling: The projects with many unknown risks that occur as the development proceeds, in that case, Spiral Model is the best development model to follow due to the risk analysis and risk handling at every phase.
2. Good for large projects: It is recommended to use the Spiral Model in large and complex projects.
3. Flexibility in requirements: Change requests in the requirements at a later phase can be incorporated accurately by using this model.
4. Customer satisfaction: Customers can see the development of the product at the early phase of the software development and thus, they habituated with the system by using it before completion of the total product.
5. Iterative and incremental approach: The Spiral Model provides an iterative and incremental approach to software development, allowing for flexibility and adaptability in response to changing requirements or unexpected events.

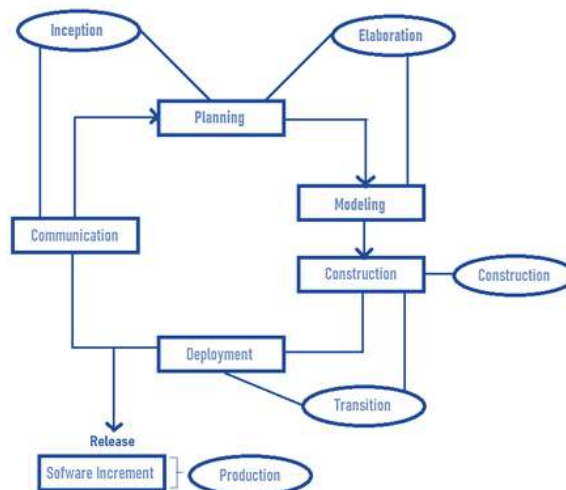
Disadvantages of the spiral model

1. Complex: The Spiral Model is much more complex than other SDLC models.
2. Expensive: Spiral Model is not suitable for small projects as it is expensive.

3. Too much dependability on risk analysis: The successful completion of the project is very much dependent on risk analysis. Without very highly experienced experts, it is going to be a failure to develop a project using this model.
4. Difficulty in time management: As the number of phases is unknown at the start of the project, time estimation is very difficult.
5. Complexity: The Spiral Model can be complex, as it involves multiple iterations of the software development process.

Unified process Model

Unified Process (UP) is characterized by its iterative and incremental approach to software development. The phases in Unified Process provide a structured framework for managing the various activities and



tasks involved in building a software system.

Inception phase

Inception phase combines the features of both communication and planning tasks of software development. Hence in this phase, end user is communicated and business requirements are collected. Later, a simple outline depicting the architecture of the projects is drawn. Here, the major requirements of the project and also the activities performed by each stakeholder in the project is represented in the form of highly abstract use cases. Hence, at this stage only fundamental details of the project are revealed. Since, the planning phase is also being considered, the developer must even work on the projects like identification of resources, major risks, the schedule of the project etc. As all the details in this form are represented in the most abstract form, it can be refined in forthcoming phases. Usage of use cases provides project scope which is essential during project planning.

Elaboration phase

The elaboration phase combines the features of customer communication and modelling tasks of the software development process. At this stage, the abstract information provided during the inception phase is refined and enlarged. The only view which existed previously is now divided into five different aspects i.e., five different models providing five different views. These models are:

- Use case model
- Analysis model
- Design model
- Implementation model
- Deployment model

In this phase, an executable architectural base line is proposed. This base line though provides viability of the project but at the same time cannot provide a complete functionality of it. Further, any modifications (if required) are also made to the project depending on the requirement.

[Discover more](#)
[Project Management Software](#)
[Projects](#)
[Programming](#)

Construction phase

Construction phase is same as the Software Development Construction. The primary aspect of this phase is to develop suitable code for each component of the software. For this purpose, various models are considered which are designed during inception and elaboration phases of a unified process. Especially, a final approval is made to the use case developed and they are transformed into code. Then unit testing is performed for each component. When this testing is done, integration of each of these components is performed. Finally, this session is concluded by conducting acceptance tests.

Transition phase

Transition phase marks the end of construction and the beginning of deployment phases respectively. Hence all the activities necessary to leave behind in construction phase completely are performed. So the attention will now be only towards the implementation of the deployment phase. During this phase, usually the software developed is adhered to the end users for a better testing. After collecting the information on the number of defects and queries related to operation of the software, well defined modifications are made to it. Also, software support information is prepared i.e., troubleshooting information, installation procedure, user guides etc., and the product is moved to the final phase.

[Discover more](#)
[Software](#)
[projects](#)
[construction](#)

Production phase

Production phase reflects the deployment activity to certain extent. Here, the software team makes a close observation of the software by exercising it through various levels. Again, any of the defects encountered are reported and are modified. Software is deployed in the machine and is checked to ensure that it remains compatible with the surroundings.

Advantages of the unified process model

- **Iterative and Incremental:**
The Unified Process is iterative, meaning that it divides the development process into iterations (phases). Each iteration results in a working version of the software. This allows for continuous feedback and better adaptability to changes.
- **Risk Management:**
Early risk identification and management are part of the process, which helps in addressing potential issues before they become significant problems.
- **Clear and Well-Defined Phases:**
The Unified Process is divided into distinct phases such as Inception, Elaboration, Construction, and Transition. This structure provides clear milestones for the project and ensures focus on different aspects of the software development life cycle (SDLC).

- **Focus on Architecture:**
Emphasizes the importance of designing a robust and scalable architecture at the beginning of the project. This helps ensure that the system can evolve smoothly over time.
- **Adaptability:**
The process can be adapted to different project sizes, complexity levels, and industries. It is not overly prescriptive, so it can fit various project needs.

Disadvantages of the unified process model

- **Complexity:**
The Unified Process can be quite complex to implement, especially for smaller projects or teams. It has a steep learning curve due to its comprehensive set of practices and frameworks.
- **Overhead:**
The detailed planning, documentation, and iterative nature of the process can result in significant overhead in terms of time and resources, particularly in smaller or simpler projects.
- **Not Ideal for Small Projects:**
For small, less complex projects, the process might be seen as too heavyweight and not efficient enough. It may be overkill for projects that don't require such an extensive methodology.
- **Requires Expertise:**
To successfully implement the Unified Process, a team requires expertise in its principles and practices. Without proper knowledge or training, teams may struggle to adhere to the methodology correctly.
- **Focus on Documentation:**
While documentation is useful, the emphasis on thorough documentation can be time-consuming and may take away from actual development, especially in dynamic environments where requirements frequently change.

[Discover more Engineering & Technology project](#)

Functional and Non-Functional Requirements

1. Functional requirements

Functional requirements define the specific behavior or functions that a system, application, or product must be able to perform. They focus on the actions the software must take to meet the user needs and business goals.

Functional requirements are often described in terms of inputs, processes, and outputs, detailing what the software should do.

Functional requirements specify what the product must do in order to satisfy the basic reason for its existence. They are

1. Specifications of the product's functionality.
2. Actions that the product must take; check, compute, record, and retrieve.
3. Derived from the basic purpose of the product.
4. Normally business oriented, rather than technical.
5. Not the technical solution constraints that are often referred as the 'system requirements
6. Derived mostly from the use case scenarios.
7. Not a quality.
8. Not measurable or testable at this stage.

2. Non-functional requirements

Non-functional requirements (NFRs) refer to the quality attributes, system characteristics, and constraints that describe how a system should behave, rather than what it should do. They focus on the performance, usability, reliability, scalability, security, and other aspects that impact the overall experience and operational efficiency of the system.

[Discover more](#)
[Dictionaries & Encyclopedias](#)
[Economics](#)
[Software](#)

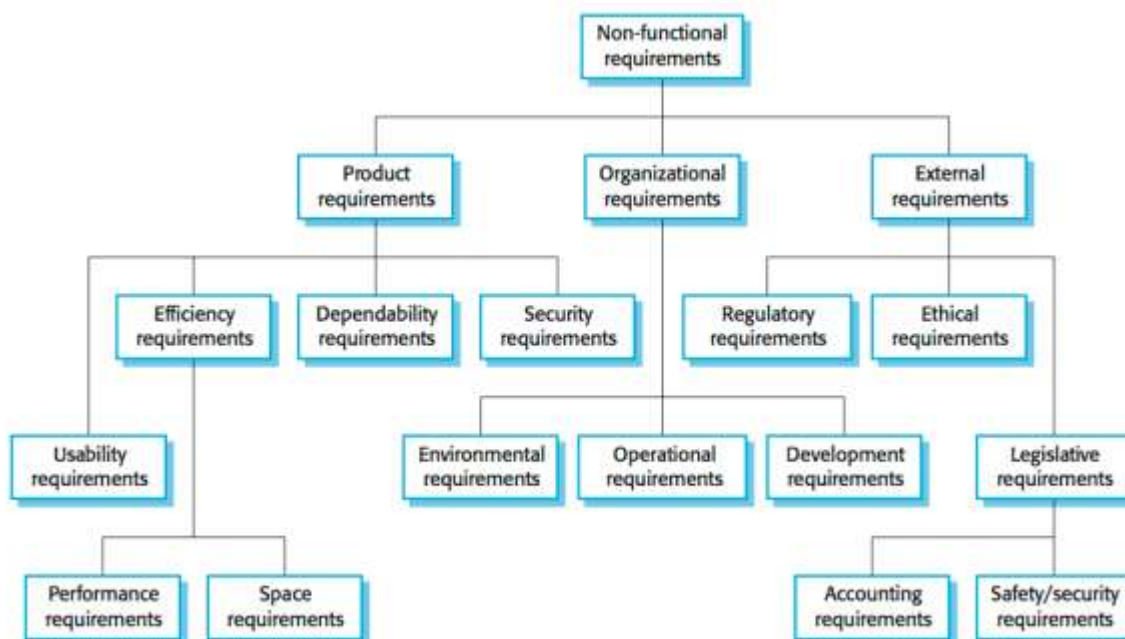
Hence, they are critical than functional requirements. Using these requirements, a specimen can easily avoid the errors occurring due to miss-implemented functional requirements. For example, no one would prefer of purchasing a water heater if its operation is unreliable i.e., if the heater is shock prone.

While dealing with non-functional requirements, one has to remember; it not only deals with the end product or the system (being developed) but also its process of manufacturing.

There are many reasons for the outcome of non-functional requirements. They are listed below:

- Budget constraints.
- Inadequate user needs.
- If there is requirement of one system to be operable over other software/hardware.
- Strong privacy and safety constraints.
- Strong organizational policies etc

Classification of non-functional requirements



Product requirements

Requirements which specify that the delivered product must behave in a particular way, example: execution speed, reliability etc.

Organizational requirements

Requirements which are a consequence of organizational policies and procedures, example: process standards used, implementation requirements etc.

[Discover more](#)
[software](#)
[Networking](#)
[Privacy Issues](#)

External requirements

Requirements which arise from factors which are external to the system and its development process, example: Interoperability requirements, legislative requirements etc.

Examples of functional and non-functional requirements

1. Online banking system

Functional requirements:

- Users should be able to log in with their username and password.
- Users should be able to check their account balance.
- Users should receive notifications after making a transaction.

Non-functional requirements:

- The system should respond to user actions in less than 2 seconds.
- All transactions must be encrypted and comply with industry security standards.
- The system should be able to handle 100 million users with minimal downtime.

2. Food delivery app

Functional requirements:

- Users can browse the menu and place an order.
- Users can make payments and track their orders in real time.

Non-functional requirements:

- The app should load the restaurant menu in under 1 second.
- The system should support up to 50,000 concurrent orders during peak hours.
- The app should be easy to use for first-time users, with an intuitive interface.

User Requirements

User requirements include both functional as well as non-functional details. The details should only provide the external behavioral aspects of the system. Hence, it should not include its implementation details which may frustrate the nontechnical users. The details should be written in terms of diagrams, forms, tables etc., to a major extent. Apart from this, there may be certain other problems which may evolve due to the usage of natural language in writing these specifications.

Few of these problems are listed below

Problem 1: Requirements amalgamation

This problem usually arises when several requirements are clubbed together and are specified as one.

Problem 2: Lack of clarity

When the requirements are not specified in a precise and unambiguous way, then clarity is said to be lacking.

Problem 3: Requirements confusion

This problem usually arises whenever the developers do not specify a clear distinction between non-functional requirements, functional requirements, design information and system goals.

[Discover more](#)
[Upgrading Desktop Operating Systems](#)
[Implementing Quality Assurance Software](#)
[Enrolling In Online Coding Courses](#)

Moreover, while writing user requirements, one has to remember that it should not be included along with the details of the system requirements. The user requirements should be precise. If it covers large amount of data then the software developer cannot include solutions to the frequently occurring problems and it becomes difficult to read. Hence, it should provide only the essential facts, while avoiding the unnecessary details. Whenever any such details are provided, a supporting reason (behind its inclusion) should be associated. This reason describes its importance and it can be considered whenever suitable changes are made to the system.

Following are certain important, principle guidelines recommended to be followed while preparing the user requirements

(i) It is a good practice to highlight the useful information.

Highlighting can be done by either making the data italic, underline, bold or by changing its colour.

[Discover more](#)

[Exploring Creative Design Agencies](#)

[software requirements](#)

[Discovering Team Productivity And Workflow Software](#)

(ii) The user requirements documents should not get mixed up with the system using the information.

(iii) While writing requirements, it is good practice to follow standard notations throughout the document.

For Example the requirements can be in bold format while reason of its inclusion in normal text.

(iv) Further, in order to make the document more realistic, the source from which the given reason is acquired can also be associated with a particular reason. This provides ease in considering the source when suitable changes are being made to these requirements.

(v) While specifying the requirements, there should be clear distinction between the mandatory and desirable requirements.

[Discover more](#)

[Accessing Technical Reference Manuals](#)

[Software](#)

[software](#)

(vi) Computer jargons must be avoided to the possible extent.

System Requirements

‘System Requirements’ is often the initial step in the system designing process. They provide the implementation details of the user requirements. Hence, they are more detailed than user requirements. The data within the system requirements should be complete as well as consistent.

System requirements should only consider the description of the operational as well as behavioral aspects of the systems. It should not include the implementation as well as the design details of the system. But, whenever large and complex systems are considered the design details are avoided in the system requirements. There are many reasons supporting this occasion. Few of them are quoted below.

If the system is distributed in nature (i.e., the system supporting interoperability), the design of a given system should be focused on the system requirements.

In order to ensure that the current system satisfies nonfunctional requirements, system architecture need to be considered.

[Discover more](#)

[Computer Hardware](#)

[software requirements](#)

[Operating Systems](#)

However, there many other factors which can create complexities while documenting the system requirements. The fundamental factor is the usage of natural language while documenting these requirements.

They not only make the text difficult to understand, but they are often confusing.

Following is a list of reasons supporting the illustration

(i)

As the requirements are specified by a developer in his own notion, it rests with the end user to properly make the distinction between the available requirements from the given volumes of text. This is because, given requirement may be grammatically analogous to other available text or given requirements may look similar but refer to two different facts etc.).

(ii)

It is often impractical to associate the requirements under different side headings. Hence, a single requirement will drive through the entire system requirements set.

(iii)

If the writer is addicted to using certain or desired phrases on different occasions, then readers may misunderstand the concept. This may deviate them from the actual requirements. Sometimes these facts are analyzed during final phases of the software development. On those occasions the costs of development will eventually rise. When these problems were analyzed by the scientists, they concluded to use a language (while writing the requirements) which can be easily grasped by all the individuals involved.

This provides a full opportunity to the writers to express their views in the most appropriate manner.

[Discover more](#)
[Computer Science](#)
[Programming](#)
[Engineering & Technology](#)

Using the language, the requirements are framed in a well planned format with suitable models used based on the requirements. The structure can also have models exchanging dialogues and other perspectives.

Following are the set of notations which can be used for the requirement specifications

Specifying system requirements

Notation: Graphical notations

Illustration: This refers to representation consisting of certain graphical objects along with appropriate text inscribed at suitable locations. Good examples of such notations are SADIT, use case diagrams and sequence diagrams.

Notation: Structured natural language

Illustration: This notation facilitates the writers to utilize certain predefined formats as well as templates while expressing a scenario in the form of requirement.

Notation: Mathematical specifications

Illustration: These notations use the mathematical concepts like sets or finite state machines. Even though these notations minimize customer – contractor arguments, customers hesitate to accept it as a system contract.

Notation: Design description language

Illustration: In this case, the writers can utilize certain programming languages, in narrating the given requirements in terms of an operational model, if the model reflects only the abstract notion of the requirement.

Structured natural language specifications

Structured Natural Language was initially proposed by Heninger with an intention of providing certain standards while writing the system requirements for an aircraft software system.

[Discover more](#)
[software engineering](#)
[software](#)
[Social Sciences](#)

According to him, structured natural language is a decent style of writing system requirements which privileges the writers with well defined standards. It also helps in curbing writers independency of using their own visualizations. Hence, there will be high degree of expressibility but this expressibility can be adhered with equal measures of uniformity.

Structured natural language helps in decreasing the number of words used in describing the context of the system. It allows to look forward for usage of templates whenever necessary. Also focus on some of these requirements (which remains extremely essential), structured natural languages highlight them. Hence, in other words, the structured natural language expresses the system requirements in certain standard forms,

While dealing with these form-based writing of system requirements, following illustrations must be considered

(i)

There should be significant description corresponding to a function or a given entity.

(ii)

Significant description corresponding to the inputs of the system as well as the source producing them. Also the description of outputs and the entities using them.

(iii)

Illustrations related to the actions to be performed.

(iv)

Illustration corresponding to generation of side effects while performing a given operation.

(v)

Illustration on initial, as well as final conditions existing before and after a given function, if the applications consisting of functions.

Domain analysis

Domain Analysis plays an important role in specifying whether the system can function properly or not. This is because domain requirements focus on the system's application domain and not on the user's and system's requirements. While performing requirement engineering, the analysis pattern is performed

repeatedly across several applications with respect to a particular business domain. If analysis pattern can be defined or categorized in a way that helps in identifying and reusing the pattern, then it is possible to create an analysis model. The impact of such a creation is that the use of reusable design pattern and executable software component increases. This in turn enhances the time-to-market and decreases the cost associated with the development of the pattern.

Domain Analysis contains the information required for recognizing, defining and categorizing the analysis pattern.

According to Firesmith, software domain analysis is defined as a process of identifying, analyzing and specifying the common reusable requirements from a specific application domain.

Firesmith also defined object oriented domain analysis as a process of identifying, analyzing and specifying the common reusable capabilities present in a specific application domain.

These capabilities are defined in terms of classes, objects and frameworks.

The specific application domain can be of wider range i.e., it basically ranges from avionics to banking or from multimedia video games to software embedded systems. The objective of domain analysis is to identify, create analysis classes, functionality that are mainly applied for making them reusable. The information about expert advice, customer surveys and existing applications are extracted from sources of domain knowledge and are provided as inputs to domain analysis. Once the given inputs are processed, the outputs generated are class taxonomies, reuse standard and domain languages. These outputs are in turn supplied as input to domain analysis model. The main purpose of using domain knowledge is to identify reusable objects across the specific application domain.

The entire process of domain analysis is characterized based on the context related to the object oriented software engineering. This process can be applied not only to the software engineering model but also to the traditional object oriented design models.

[The following are the steps carried out while performing the domain analysis process](#)

1. The domain that needs to be analyzed must be defined.
2. The items retrieved from sources of domain knowledge must be categorized.
3. The sample of application defined in the domain must be collected.
4. Every individual application in the sample is analyzed. Based on this, analysis class must be defined.
5. An analysis model forth class must be developed.

Interface Specification

Software systems must be able to work with systems that are already implemented and installed. Thus, interfaces between them must be precisely specified. Specifications related to interfaces must be included in the requirements document as early as possible.

Types of interfaces

1. Procedural interfaces

Application Programming Interfaces (API) are the procedural interfaces. Any service offered by a program or sub-system can be accessed by invoking an interface procedure.

2. Data structures

A sub-system may pass data structures to another sub-system. To describe these structures, Java or C++ automatically generated descriptions or graphical data models are used.

3. Representations of data

The way data is represented, for example, bit ordering, can be used as an interface. Most of the real-time embedded systems use these interfaces. Moreover, Ada also supports them.

[Discover more](#)
[Software Requirements](#)
[Engineering & Technology](#)

Software requirements document

1. Both customer as well as system requirements specifications can be integrated to form one document.
(or)

Customer requirements specification can be provided in the initial phase or at the introduction of the document. This step is followed by detailed system requirement specification.

(or)

If the application is complex and probably large then system requirement specifications can be included at separate locations or document. Usually, in each software requirements document the details of large sets of stakeholders involved in the development are included.

Following is a list of stakeholders and illustrations related to their involvement in the development,

(a) System customer and end user

These are the people to whom software is being delivered. They are authenticated to supply their requirements and also read the current document (software requirement document) in order to ensure that the (document) is built in accordance to their needs.

(b) System managers

System managers usually consider the document to decide on the cost of development and to plan out the consequences of entire project.

(c) Development team or engineers

They usually decide on the tasks to be accomplished.

(d) Testers

They use the documents in order to develop suitable test cases.

(e) Maintenance engineers

They consider the document as a mode to understand as well as generalize the relationship existing between various modules of the system.

[Discover more](#)
[Software Requirements](#)
[Technical Reference](#)
[Software](#)

In general, the level of details to be provided in the document depends on the type of system being developed and also on the process being implemented on this occasion.

By considering various aspects, IEEE has launched following format for generating the software requirements documents,

1. Introduction

- Purpose of requirements document
- Product scope
- Definitions, acronyms and abbreviations
- References
- Overview of rest of the requirements document

2. General description

- Product perspective
- Product functions
- User characteristics
- General constraints
- Assumptions and dependencies

3. Specific requirements

- Functional
- Non-functional
- External interface requirements

- System functionality and performance
- Logical database requirements
- Design constraints
- Emergent system properties
- Quality characteristics

4. Appendices

Here, the details of the application are included. The fundamental issues which can be addressed in this regard are hardware requirements as well as database requirements.

5. Index

Indexes are usually included to provide ease in locating appropriate entities. The most fundamental index is given in alphabetical order. There are many ways to represent indexes and any of them can be followed. The above mentioned format is a standard notation for developing the software requirement specification. But it is assumed that the above format defines only general characteristics when considered along with the organization developing the software. Hence, the above format is further in matured and certain new specifications are added to it. This helps in directly applying preparation of SRS at the organization level. Hence, the improved format is given below,

[Discover more](#)
[software requirement](#)
[Engineering & Technology](#)
[software](#)

1. Preface

Here, the fundamentals of the document are defined (say) the current version and its history. Also the details include the reasons for introducing the current documents along with the summary of changes made to the previous version in bringing up the current version.

2. Introduction

Summary of details under this side heading are given below,

The need for the system

The major functionality within the system and with other systems.

The way the given system remains compatible with the business objectives of the organization involved in developing the given system

3. Glossary

Glossary includes definitions for various technical terms used in requirements document.

4. User requirements definition

Here, usually the services delivered by a given system to the users are specified along with non-functional requirements of the system. This illustration is usually made in most elucidating style using the formal language constructs, diagrams or user understandable notations.

5. System architecture

Here, usually the overall system architecture is presented in an abstract format. This includes the distribution of modules throughout the architecture along with the relationships existing between them. The components (within the architecture) which are used more than once, should be represented along with the same signature (representation) so that they can be recognized easily.

6. SRS or System requirement specification

In this case both functional as well as nonfunctional requirements are described in a more detailed manner.

[Discover more software requirements software requirements document Business & Productivity Software](#)

7. System models

One or more models depicting the relationship between the system, its environment and its components, along with their vicinity where they are highlighted. The most commonly used models in this case are semantic, data, dataflow and object models.

8. System evolution

In this case usually the assumptions based on which the current system is brought up is summarized. Also, the probable changes which can be made to the system on the account of hardware trends and future user needs are also addressed.

9. Appendices

Here, the details of the application are included. The fundamental issues which can be addressed in this regard are hardware requirements and database requirements.

10. Index

Indexes are usually included to provide ease in locating appropriate entities. The most fundamental index is given in alphabetical order. There are many ways to represent indexes and any of them can be followed.

Characteristics of a good software requirement specification

A good SRS document must possess the following characteristics

(i) Concise

The SRS document must be up to the point, unique and consistent because lengthy and irrelevant descriptions decrease the readability and increase the possibility of errors.

(ii) Organized

The SRS document must be well-organized because it becomes simple to understand and one can make the necessary changes easily as per the customer requirements.

(iii) Black-box view

The SRS document should represent only the physical behaviour of the system rather than describing its implementation issues. In other words the system which is being developed must be considered as a black box and its external behaviour should be defined. Due to this reason, the SRS document is also known as black-box specification of a system.

(iv) Conceptual integrity

A good SRS document must possess conceptual integrity so that the contents of the document can be understood easily by a reader.

(v) Response to undesired events

The document must describe the system responses to exceptional conditions.

(vi) Verifiable

All requirements of the system must be verified in order to determine whether all the requirements are met during implementation or not. If any feature is not verifiable then it should be mentioned separately in the “goals of implementation” section of the SRS document.

Role of software requirement specification in software engineering

Software Requirement Specification (SRS) document is the final outcome that comes after problem analysis. But, both problem analysis and SRS are carried out simultaneously due to which these activities overlap with movement from both the activities to the other. The information regarding the SRS is obtained from analysis therefore, specification activity follows the analysis activity.

The formal modeling performed in problem analysis is not treated as SRS as it lays stress on the problem structure than the external behaviour. The things like modeling of user interfaces, minor issues, performance, design constraints, recovery etc., are rarely modeled. But these things should be specified in SRS, these all constraints are needed to be modeled to design a system easily and clearly. On the other hand, it is difficult for some structures to be translated into external behaviour specification due to their limit use.

Software Requirement Specification (SRS) Format

In order to form a good SRS, here you will see some points that can be used and should be considered to form a structure of good Software Requirements Specification (SRS). These are below mentioned in the table of contents and are well explained below.

Software Requirement Specification (SRS) Format as the name suggests, is a complete specification and description of requirements of the software that need to be fulfilled for the successful development of the software system. These requirements can be functional as well as non-functional depending upon the type of requirement. The interaction between different customers and contractors is done because it is necessary to fully understand the needs of customers.

Document Title

Author(s)

Affiliation

Address

Date

Document Version

Depending upon information gathered after interaction, SRS is developed which describes requirements of software that may include changes and modifications that is needed to be done to increase quality of product and to satisfy customer's demand.

Introduction

- **Purpose of this Document** - At first, main aim of why this document is necessary and what's purpose of document is explained and described.
- **Scope of this document** - In this, overall working and main objective of document and what value it will provide to customer is described and explained. It also includes a description of development cost and time required.
- **Overview** - In this, description of product is explained. It's simply summary or overall review of product.

General description

In this, general functions of product which includes objective of user, a user characteristic, features, benefits, about why its importance is mentioned. It also describes features of user community.

Functional Requirements

In this, possible outcome of software system which includes effects due to operation of program is fully explained. All functional requirements which may include calculations, data processing, etc. are placed in a ranked order. Functional requirements specify the expected behavior of the system-which outputs should be produced from the given inputs. They describe the relationship between the input and output of the system. For each functional requirement, detailed description all the data inputs and their source, the units of measure, and the range of valid inputs must be specified.

Interface Requirements

In this, software interfaces which mean how software program communicates with each other or users either in form of any language, code, or message are fully described and explained. Examples can be shared memory, data streams, etc.

Performance Requirements

In this, how a software system performs desired functions under specific condition is explained. It also explains required time, required memory, maximum error rate, etc. The performance requirements part of an SRS specifies the performance constraints on the software system. All the requirements relating to the performance characteristics of the system must be clearly specified. There are two types of performance requirements: static and dynamic. Static requirements are those that do not impose constraint on the execution characteristics of the system. Dynamic requirements specify constraints on the execution behaviour of the system.

Design Constraints

In this, constraints which simply means limitation or restriction are specified and explained for design team. Examples may include use of a particular algorithm, hardware and software limitations, etc. There are a number of factors in the client's environment that may restrict the choices of a designer leading to design constraints such factors include standards that must be followed resource limits, operating environment, reliability and security requirements and policies that may have an impact on the design of the system. An SRS should identify and specify all such constraints.

Non-Functional Attributes

In this, non-functional attributes are explained that are required by software system for better performance. An example may include Security, Portability, Reliability, Reusability, Application compatibility, Data integrity, Scalability capacity, etc.

Preliminary Schedule and Budget

In this, initial version and budget of project plan are explained which include overall time duration required and overall cost required for development of project.

Appendices

In this, additional information like references from where information is gathered, definitions of some specific terms, acronyms, abbreviations, etc. are given and explained.

Uses of SRS document

- Development team require it for developing product according to the need.
- Test plans are generated by testing group based on the describe external behaviour.
- Maintenance and support staff need it to understand what the software product is supposed to do.
- Project manager base their plans and estimates of schedule, effort and resources on it.
- customer rely on it to know that product they can expect.
- As a contract between developer and customer.

QUESTION BANK:

- 1. What is software engineering? Explain the role of software engineering. (8 Marks)**
- 2. Describe in details a layered technology in software engineering. (8 Marks)**
- 3. Differentiate Functional and non-functional requirements. (8 Marks)**
- 4. Explain the term user requirement and system requirement. (8 Marks)**
- 5. Define software engineering. What are software myths explain in detail. (8 Marks)**
- 6. Define process model. Explain increment process model in details. (8 Marks)**
- 7. How does the Capability maturity model integration (CMMI) improve the business process in an IT company. (8 Marks)**
- 8. Explain personal and team process models. (8 Marks)**
- 9. Explain the framework activities in spiral model with suitable diagram. (8 Marks)**
- 10. Explain software requirement documents. (4 Marks)**
- 11. Explain software evolution. (4 Marks)**
- 12. Discuss the software requirement documents. (4 Marks)**